Sprint 7 Documentation – Integration Test, Batch Processing of Access Logs

Team: Sebastian and Felix

Repository: https://github.com/BunnySupreme/SWEN3Project

Sprint duration: 09.01.2026 – 18.01.2026

Estimated workload: 8 h / person / week

1. Sprint Goal (from assignment)

1. Show the functionality of use case “document upload” with an integration test
2. Create an additional application for running the scheduled service that reads daily XML files from an input folder to process access logs from external systems via a batch process

2. Implemented Work

Integration Test

The integration test checks if the use case “document upload” succeeds step-by-step, alongside other related tasks of the Paperless application:

1. auth_register_if_needed: Attempts registering a new user, which is a required step for uploading documents, as only registered users may upload
2. auth_login: Attempts logging in with the registered credentials
3. test_upload_happy_path: Attempts to upload a valid document using the relevant endpoint
4. test_upload_invalid_file: Attempts to upload an invalid document using the relevant endpoint (this should fail)
5. test_upload_without_file: Attempts to upload nothing using the relevant endpoint (this should fail)
6. test_worker_log_for_upload: Checks whether the OcrWorker’s logs indicate processing of files by the worker service by uploading another valid file and comparing logs before and after upload
7. test_login_wrong_password: Attempts to log in with incorrect credentials (this should fail)
8. test_protected_endpoint_requires_auth: Attempts to call an endpoint requiring authentication without an auth token (this should fail)
9. test_protected_endpoint_rejects_bad_token: Attempts to call an endpoint requiring authentication with an invalid auth token (this should fail)
10. test_logout_revokes_token: Checks whether the auth token is revoked upon logout

11. test_search_documents: Attempts to search for a specific search term, includes auth tests and attempts at searching without a search term (the latter should fail)
12. test_summary_filled_after_upload: Checks whether a summary was created for an uploaded document

Batch Processing of Access Logs

Two processes run in tandem. Frontend and REST server create access logs:

1. Users view the frontend document details page (DocumentDetail) of a document, which also contains the option to download the document
2. This triggers a POST-Request sent to the REST server (endpoint: api/accesslog/<documentId>/doc-access), handled by the AccessLogController
3. The controller calls upon XmlService's UpdateAsync method to create or update today's XML with the access

Meanwhile the Batch Processing application (a microservice) checks for XMLs at regular intervals (every minute for test purposes, would be daily in a productive environment):

1. BatchWorkerService in regular intervals calls upon XmlProcessorService to execute the RunOnceAsync method
2. RunOnceAsync checks if XMLs are present in the input directory
3. If so, it reads it, tells the DocumentRepository to execute UpdateAccessCountAsync (which updates the document's access count in the DB), and moves the XML to the archive directory (or to the error directory, should the process fail)

3. Workload (Estimate)

Team Member	Week / Sprint	Estimated Hours	Main Tasks
Sebastian	Sprint 7	8	Batch Processing of Access Logs, Documentation
Felix	Sprint 7	8	Integration Test, Documentation